

Design and Realization of a Benchmarking Testbed for Evaluating Autonomous Platooning Algorithms

Michael H. Shaham, Risha Ranjan, Engin Kırda, and Taşkın Padır

Northeastern University, Boston, MA 02115, USA,
shaham.m@northeastern.edu,
<https://robot.neu.edu/>

Abstract. Autonomous vehicle platoons present near- and long-term opportunities to enhance operational efficiencies and save lives. The past 30 years have seen rapid development in the autonomous driving space, enabling new technologies that will alleviate the strain placed on human drivers and reduce vehicle emissions. This paper introduces a testbed for evaluating and benchmarking platooning algorithms on 1/10th scale vehicles with onboard sensors. To demonstrate the testbed’s utility, we evaluate three algorithms, linear feedback and two variations of distributed model predictive control, and compare their results on a typical platooning scenario where the lead vehicle tracks a reference trajectory that changes speed multiple times. We validate our algorithms in simulation to analyze the performance as the platoon size increases, and find that the distributed model predictive control algorithms outperform linear feedback on hardware and in simulation.

1 Introduction

1.1 Motivation

Autonomous driving technologies have the potential to improve safety, reduce fuel emissions [20], reduce stress, and more [21]. In this work, we focus on autonomous vehicle platoons, or groups of autonomous vehicles driving together, which also have the potential to increase highway capacities and reduce travel time. Since the 1990s, there have been numerous efforts to demonstrate platooning technologies including the California PATH program demonstration [25], the 2011 Grand Cooperative Driving Challenge [18], the Energy ITS project [26], and the SARTRE platooning program [20]. Though these demonstrations highlight the feasibility of platoons for the trucking industry or automated highway systems, they often evaluate only a single algorithm, and there is still a need to benchmark algorithms to compare safety and performance [17].

Validation platforms for autonomous driving algorithms on hardware have been introduced, but these systems typically focus on autonomous racing [9, 16, 27] or coordinated planning and networked control [8, 10]. With the exception of [11] which focuses on heavy-duty vehicles, there remains a lack of focus on vehicle

platooning. This work introduces a research platform that builds on the widely used open source F1Tenth system [16] to test autonomous platooning algorithms. Using our platform, we evaluate three different algorithms and perform further analysis of these algorithms in simulation.

1.2 Related Work

The platooning problem was studied as early as the 1960s [12], but it was not until the California PATH demonstration and associated research in the 1990s when platooning began to receive considerable attention [25]. Since then, many theoretical developments have provided insight into the performance of vehicle platooning algorithms. These insights typically relate to how the information flow topology, distributed controller, and spacing policy contribute to the overall behavior [13].

For analysis of platoons, there are two common performance metrics used: string stability [7, 24] and (asymptotic) stability of the platoon. String stability ensures disturbances, typically due to the head vehicle’s motion, will not propagate down the string of vehicles in the platoon. An algorithm that is stable follows the usual control theoretic definition: subject to some disturbance, the vehicles in the platoon will eventually be able to reach zero error. Platoons typically use two different inter-vehicle spacing policies: (1) constant distance headway (CDH) where each vehicle maintains a constant distance to its predecessor and (2) constant time headway (CTH) where each vehicle maintains a distance based on the time required to reach its predecessor.

Linear feedback controllers have been widely used to obtain insightful theoretical results. In [28], the authors analyze the effect of the information flow topology on the stability of a platoon using a linear feedback controller and a CDH policy, and show how performance degrades significantly as platoon size increases. The type of controller along with the spacing policy can also significantly impact the performance of a platoon. In [22], authors prove that the combination of a linear feedback policy, a predecessor follower topology (each vehicle has access to information only from the preceding vehicle), and a CDH policy leads to string instability. However, under the same conditions but with a CTH policy, string stability is guaranteed [15]. Similarly, in [1], the decentralized structure of the platooning problem and linear quadratic regulator theory are exploited to design a string stable linear feedback controller using a CTH policy.

By assuming vehicles can communicate with one another, it is possible to design distributed model predictive control (DMPC) algorithms. In [5], the authors present the first DMPC algorithm that guarantees both asymptotic stability and string stability. Building on this, the authors in [29] guarantee stability (but not string stability) of the platoon without following vehicles knowing a priori the desired set point (velocity) of the platoon. Both [5] and [29] assume vehicles share their planned trajectory with neighboring vehicles, which is used at the next timestep. More recently, [19] develop a DMPC method that allows vehicles to share their true optimal planned trajectory multiple times at each timestep, while also using collision constraints to guarantee safety.

1.3 Problem Statement

A platoon consists of a string of vehicles following one another. The goal of the platoon is to have each vehicle drive at the same speed as the leader while maintaining a desired spacing relative to its neighbors (we focus on a CDH policy). Most platooning research in the literature contribute theoretical results and a simulation experiment to validate the theory, without performing any validation on hardware (and often without comparing performance to other algorithms). To evaluate platooning controllers, a benchmarking testbed needs to be developed to compare results and analyze advances and trade-offs. The testbed should use vehicles equipped with sensors that provide inter-vehicle sensing and communication. The vehicles must also be capable of following trajectories typical of highway-driving scenarios and drive at a sufficiently large range of speeds to track these trajectories.

2 Technical Approach

2.1 Hardware

The vehicles (Figure 1) used in our experiments are modified versions of the standard F1Tenth vehicle [16]. We introduce the following modifications: using an Intel NUC instead of an NVIDIA Jetson Xavier NX for faster compute, adding an AS5047P magnetic motor position sensor for better low-speed performance, using fixed suspension rods to reduce jitter due to the suspension, and adding a Pozyx ultra-wideband (UWB) radio frequency device to obtain inter-vehicle distance measurements. The Pozyx UWB device is reported to be accurate up to ± 10 cm. In our experiments, we have found its error to be roughly Gaussian with a standard deviation of 4.5 cm. All perception, planning, and control computations are completely decentralized and are performed on the Intel NUC using ROS 2 [14]. The testing environment used for hardware experiments is a 10.2 m $\times$ 6.6 m reconfigurable, oval race track.

Since this paper focuses on multi-vehicle longitudinal control, we briefly summarize the local planning each vehicle performs for lateral control. Each vehicle is equipped with a 2D lidar—either an RPLiDAR S2 (follower vehicles) or a Hokuyo UST-10LX (leader vehicle). The lidar provides data in the form of a pointcloud, which we segment using DBSCAN [6] to find the sets of points corresponding to the left and right walls. The points associated with the left and right walls are then used to estimate a polynomial centerline (in coordinates relative to the local vehicle’s frame), which is passed as input to the pure pursuit algorithm [2]. Note that all planning is performed locally, *i.e.*, the vehicles have no notion of global position within the course; they only know their relative position to their predecessor in the platoon.

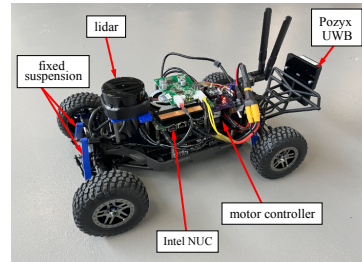


Fig. 1. One of the vehicles used in our platoon, which is a modified version of the F1Tenth vehicle.

2.2 Control Algorithms

The platoon consists of $N + 1$ vehicles, indexed by $0, 1, \dots, N$, where vehicle 0 is the leader. We analyze and compare a linear feedback and two DMPC controllers using a predecessor follower topology, depicted in fig. 2. We use a constant spacing policy, and the goal of each following vehicle is to match the velocity of and maintain a constant distance to its predecessor, *i.e.*,

$$\begin{aligned} p_i(t) &\rightarrow p_{i-1}(t) - d_{\text{des}} \\ v_i(t) &\rightarrow v_{i-1}(t) \end{aligned} \quad \text{as } t \rightarrow \infty, \text{ for all } i = 1, \dots, N \quad (1)$$

where p and v are position and velocity, respectively, and d_{des} is the desired inter-vehicle distance. Equation (1) guarantees asymptotic stability—we do not focus on string stability in this work. Note that if eq. (1) is achieved, each vehicle also matches the desired speed of and the desired spacing relative to the leader.



Fig. 2. Predecessor follower topology. The directed edge $i \rightarrow j$ indicates vehicle j has access to (or receives information from) vehicle i . Vehicles are indexed by $0, 1, \dots, N$.

Vehicle dynamics. We model each vehicle's dynamics as

$$\begin{aligned} x_i(k+1) &= A_i x_i(k) + B_i u_i(k) \\ A_i &= \begin{bmatrix} 1 & \Delta t \\ 0 & 1 - \frac{\Delta t}{\tau_i} \end{bmatrix} \quad B_i = \begin{bmatrix} 0 \\ \frac{\Delta t}{\tau_i} \end{bmatrix} \quad i = 0, \dots, N \end{aligned} \quad (2)$$

where k is the discrete sampling instance, Δt is the discretization time and τ_i is the inertial delay of the longitudinal dynamics. The state $x_i = (p_i, v_i)$ is the position and velocity and the control input u_i is the desired velocity. It is more common to use a dynamics model where the state x_i includes the vehicle's acceleration and the input u_i is the desired acceleration. However, we have found the F1Tenth vehicles perform better when specifying a desired velocity (corresponding to motor revolutions per minute) instead of an acceleration (corresponding to motor current).

Linear feedback algorithm. In the linear feedback case, each vehicle measures or receives the distance to and velocity of its predecessor. The distance measurements are obtained via the UWB device and the speed measurements are shared using ROS 2 and Wi-Fi. Real vehicles are commonly equipped with radar or a similar sensor to estimate the distance and speed of neighboring vehicles. The desired velocity input for the linear feedback controller is selected using

$$u_i(t) = k_p(p_{i-1}(t) - p_i(t) + d_{\text{des}}) + k_v(v_{i-1}(t) - v_i(t)), \quad i = 1, \dots, N$$

where $k_p, k_v > 0$ are predefined gains. See [28] for further details and stability analysis of the continuous-time version of this controller when using the typical acceleration-input version of the dynamics. The results in [28] can be extended to systems using the dynamics given by eq. (2).

Distributed model predictive control algorithms. For the DMPC controllers, at each timestep t , each vehicle receives the assumed trajectory of its predecessor, which is calculated at the previous timestep $t - 1$. Let $x_i^a(0), \dots, x_i^a(H)$ denote the assumed trajectory over the planning horizon H for vehicle i at timestep t . Then at each timestep t , the DMPC controller solves

$$\begin{aligned} & \underset{u_i^p, x_i^p}{\text{minimize}} \quad \sum_{k=0}^{H-1} l_i(x_i^p(k), u_i^p(k), x_i^a(k), x_{i-1}^a(k)) \\ & \text{subject to} \quad x_i^p(0) = x_i(t) \\ & \quad x_i^p(k+1) = A_i x_i^p(k) + B_i u_i^p(k), \quad k = 0, \dots, H-1 \\ & \quad |v_i^p(k+1) - v_i^p(k)| \leq \Delta t \cdot a_{i,\max}, \quad k = 0, \dots, H-1 \\ & \quad v_{i,\min} \leq v_i^p(k) \leq v_{i,\max}, \quad k = H, \dots, H \\ & \quad x_i^p(H) = x_{i-1}^a(H) - \tilde{d}_{\text{des}} \\ & \quad u_i^p(H-1) = v_{i-1}^a(H) \end{aligned} \quad (3)$$

where $\tilde{d}_{\text{des}} = (\tilde{d}, 0) \in \mathbb{R}^2$. This is the same formulation proposed in [29] but with slightly altered constraints to account for the dynamics model we use.

We analyze two different options for the cost function l_i . The first uses the squared weighted 2-norm to penalize errors:

$$l_i(k) = \|x_i^p(k) - x_i^a(k)\|_{F_i}^2 + \|x_i^p(k) - x_{i-1}^a(k) + \tilde{d}_{\text{des}}\|_{G_i}^2 + \|u_i^p(k) - \tilde{u}\|_{R_i}^2 \quad (4)$$

where $F_i, G_i, R_i \succ 0$ and $\|z\|_Q^2 = z^\top Q z$. The second option uses the 1-norm:

$$l_i(k) = s_i \|x_i^p(k) - x_i^a(k)\|_1 + q_i \|x_i^p(k) - x_{i-1}^a(k) + \tilde{d}_{\text{des}}\|_1 + r_i \|u_i^p(k) - \tilde{u}\|_1 \quad (5)$$

where $s_i, q_i, r_i > 0$ are scalar variables weighting each objective. The first term in these cost functions, which we refer to as the *move suppression term*, penalizes the deviation between vehicle i 's predicted and assumed trajectories. The second term, which we refer to as *predecessor relative error term*, penalizes the error between vehicle i 's predicted trajectory and its predecessor's assumed trajectory. We will refer to the DMPC algorithm (3) using eq. (4) as *squared 2-norm* (or $\|\cdot\|_2^2$) DMPC and the algorithm using eq. (5) as *1-norm* (or $\|\cdot\|_1$) DMPC. Note that the timestep-shifted version of the optimal trajectory computed at timestep t is used as the assumed trajectory for timestep $t + 1$. See [29] for further details about the implementation of the algorithm.

The inequality constraints of eq. (3) enforce velocity and acceleration limits. The equality constraints of eq. (3) serve two purposes: (1) ensure dynamic feasibility of the trajectory and (2) enforce terminal constraints to ensure recursive feasibility of the DMPC controller. If we denote the solution of this optimization problem as u_i^* , then the DMPC controller implements $u_i^*(0)$ and repeats this process at the next timestep after receiving the assumed trajectory of its predecessor. Since our controller uses the desired velocity as the input, we use $\tilde{u} = v_i(t)$, so each vehicle penalizes velocity input that deviate from its current velocity.

The DMPC algorithm we use is based on the formulation used in [29], which uses the cost function

$$l_i(k) = \|x_i^p(k) - x_i^a(k)\|_{F_i} + \left\| x_i^p(k) - x_{i-1}^a(k) + \tilde{d}_{des} \right\|_{G_i} + \|u_i^p(k) - \tilde{u}\|_{R_i}. \quad (6)$$

We will refer to this as 2-norm DMPC. The theoretical basis for using the weighted 2-norm, which is provided in [29], is given in the following theorem, which is a slight adaptation of theorem 5 from [29].

Theorem 1. *Suppose a platoon uses a predecessor follower topology, and each vehicle has dynamics given by eq. (2). If each vehicle uses the DMPC controller that requires solving eq. (3) with the cost function given by eq. (6), then the platoon's dynamics are asymptotically stable (i.e., eq. (1) is achieved) if*

$$F_i \succeq G_{i+1}, \quad i = 0, 1, \dots, N-1.$$

Recall that F_i is related to the move suppression term of the i th vehicle, and G_{i+1} is the predecessor relative error term of the i th vehicle's predecessor. Intuitively, this result says that each vehicle must place at least as much weight on maintaining its assumed trajectory as its predecessor does. Theorem 1 was derived for a nonlinear vehicle dynamics model that is equivalent to a linear dynamics model (where the state includes the acceleration) after using the feedback linearization technique. However, it is not too difficult to adjust the proof to work for the dynamics given by eq. (2) with the constraints in eq. (3).

Though not formally analyzed in [29], it is relatively easy to extend theorem 1 to the case of the 1-norm cost function given by eq. (5). The proof in [29] requires only the use of norm inequalities, and thus also applies to the case of the 1-norm, but with minor adjustments to handle the scalar terms instead of the weight matrices. With that said, we extend theorem 1 to the case of the 1-norm DMPC with the following sufficient condition to guarantee platoon asymptotic stability:

$$s_i \geq q_{i+1}, \quad i = 0, 1, \dots, N-1. \quad (7)$$

So far, we have not been able to determine any sufficient conditions that prove the DMPC algorithm with the squared 2-norm cost function (4) is stable. Our results, however, will show that the algorithm performs well in practice, so we believe it is worthwhile to consider even if it is not (yet) provably stable. Further, we decide to use the cost functions given by eq. (4) and eq. (5) as opposed to the eq. (6) because these cost functions lead to optimization problems that can be formulated as a quadratic program (QP) or a linear program (LP), respectively. These formulations allow us to plan over a much larger time horizon ($H = 100$ in our experiments) and alleviate feasibility issues due to the terminal equality constraints. For the weighted 2-norm cost function (6), we were unable to scale the time horizon above roughly $H = 20$, and ran into feasibility issues when testing on hardware where the controller needs to run at a rate of at least 10 Hz. To solve the optimization problems, we used CVXPY [3] with OSQP [23] (for the QP) or with ECOS [4] (for the LP). With these tools, we were able to solve the optimization problem (3) at rates faster than 100 Hz.

3 Experimental Results and Insights

3.1 Hardware results

To compare the performance of the two DMPC controllers and the linear feedback controller, we conducted an experiment where the lead vehicle starts at 0 m/s velocity, accelerates to 2 m/s, twice alternates accelerating to 3.5 m/s and back down to 2 m/s, before finally coming to a stop. The experiment was designed to test how well the platoon handles sudden changes in speed (*e.g.*, as in highway driving due to speed limit changes) and its ability to follow the leader from start to stop. Figure 3 shows results from this experiment with four vehicles (one leader and three followers) for each of the three controllers. We used a longitudinal time delay of $\tau_i = 0.3$ for each vehicle (determined experimentally using a least-squares regression on experimental data) and a desired spacing of 1 m. For each of the DMPC algorithms, we used the identity matrix (weighting matrices in eq. (4)) or the scalar 1 (scalar weights in eq. (5)) for all controller parameters, which satisfies conditions given in theorem 1 and eq. (7). For linear feedback, we used $k_p = 1$ and $k_v = 2$, as we found these values gave the smoothest vehicle trajectories during testing.

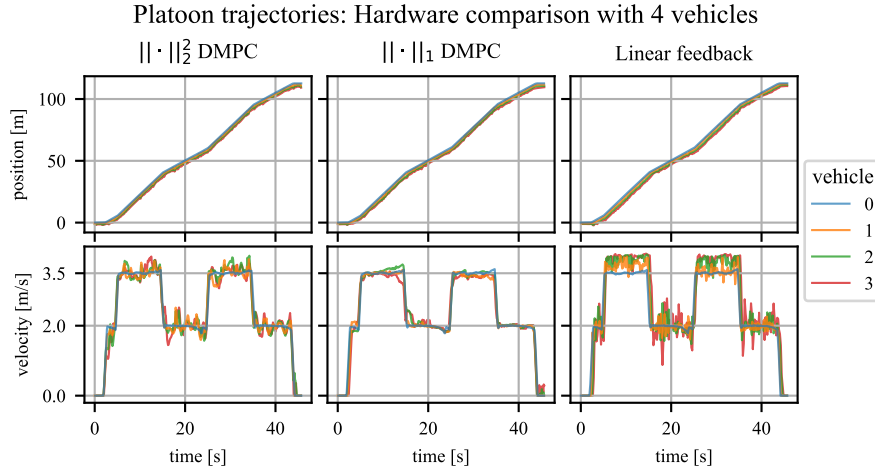


Fig. 3. Results depicting the four-vehicle platoon trajectories when using three different algorithms on the hardware described in section 2.1. The columns show the performance of the squared 2-norm DMPC, 1-norm DMPC, and the linear feedback algorithms. The position of the lead vehicle was calculated by integrating its velocity, and the positions of the following vehicles was calculated using the preceding vehicle's position offset by the distance measurement from the Pozyx UWB.

To quantitatively compare the performance between each of the three controllers, we repeated the experiment shown in fig. 3 10 times for each controller, completing a total of 30 experiments. We use the root mean square error (RMSE) of the platoon's spacing and velocity trajectories to compare the three controllers. The spacing and velocity errors are given by $p_i - p_{i-1} + d_{\text{des}}$ and $v_i - v_{i-1}$,

respectively. For each experiment, we calculate the mean spacing and velocity RMSE for each following vehicle. We then calculated the mean and standard deviation of these mean RMSE values over the ten experiments, and calculated a 95% confidence interval for the mean RMSE of each vehicle’s position and velocity. The results are shown in fig. 4. As we can see, the two DMPC controllers clearly outperform the linear feedback controller.

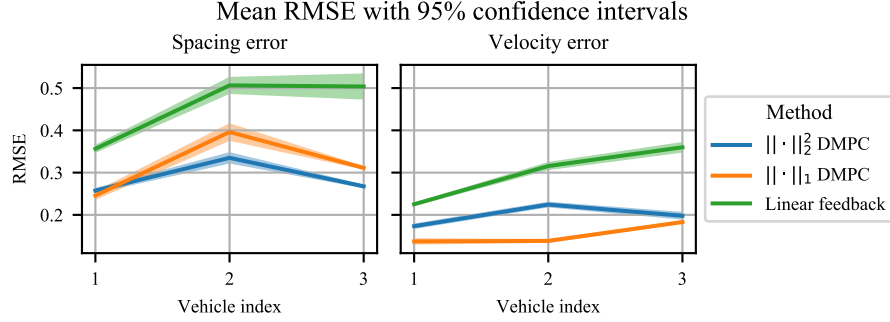


Fig. 4. A comparison of results over ten trials of each algorithm.

In terms of spacing error, the squared 2-norm DMPC performs better than the 1-norm DMPC for the two furthest followers, and the two algorithms have comparable performance for the first follower. The 1-norm DMPC controller performed the best in terms of velocity error. One thing our analysis on hardware is unable to tell us is how well these algorithms scale as the size of the platoon increases. To investigate this issue, we conducted experiments in simulation, discussed below.

3.2 Simulation results

To analyze how well these algorithms perform as the size of the platoon increases, we simulate a platooning experiment with $N = 100$ following vehicles. In the experiment, the leader follows a reference trajectory that starts at 20 m/s, accelerates to 25 m/s and waits for 30 seconds, then decelerates back down to 20 m/s. Trajectory results for a few vehicles are shown in fig. 5. We use $\tau_i = 0.3$ for each vehicle and set the desired spacing at 5 meters. We also used the same controller parameters that were used in the hardware experiments for the simulation study. For all simulation experiments, we add zero-mean Gaussian noise with a covariance of $.3I$ to the dynamics and with a standard deviation of 4.5 cm to the inter-vehicle spacing sensing (to match the error encountered by the Pozyx UWB device).

Figure 5 shows that the two DMPC algorithms are able to remain close to the desired platoon trajectory without requiring unsafe trajectories. The 100th vehicle in the platoon for each of these controllers reaches a peak velocity of almost 28 m/s before settling to the desired velocity of 25 m/s during the acceleration portion of the experiment. They also perform well when decelerating.

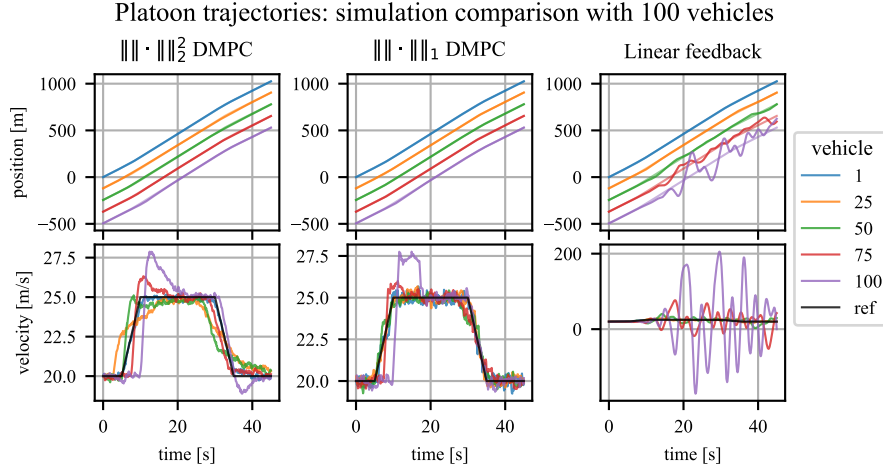


Fig. 5. Results from one simulated experiment with a 100-vehicle platoon. The reference velocity trajectory the leader follows is shown as the black curve in the velocity plots (bottom row). In the position plots, the vehicle 1 trajectory is indistinguishable from the reference. Note the difference in y -axis scales for the bottom row of plots. We also plot the desired positions of each vehicle relative to the leader as faded curves in the top row, and this is most visible in the linear feedback case.

Despite this, it is still clear that the performance relative to the true desired goal of maintaining the leader’s velocity profile degrades as we move further down the platoon (since platoons further from the leader need to achieve higher velocities to maintain performance). Though it is not shown in these figures, when using the DMPC controllers, each of the vehicles maintained a spacing error of less than roughly one meter, meaning the vehicles never came close to crashing when using a desired spacing of 5 meters. It is not clear to what extent we would need to scale the platoon size N before seeing a collision between two vehicles under this problem setup.

The performance of the linear feedback algorithm, however, quickly degrades as the platoon size increases. Though we saw that performance on hardware was reasonable, we can see from the position trajectories that if we continue to allow the platoon size to increase, the vehicles will crash if the desired spacing is not large enough. Note that we allowed the linear feedback vehicles to apply any unconstrained input and set no bounds on the state. Thus, the 100th vehicle in the platoon was allowed to command large control inputs in an attempt to stabilize. Distributed linear feedback controllers using a CTH policy would alleviate the issues we see here, but we do not investigate this problem setting in this work.

Similar to the hardware experiments, we repeat the simulation experiment ten times and investigate the average performance of each vehicle in the platoon with respect to RMSE. Figure 6 shows the results of this analysis. In line with the results in [28], the performance of linear feedback quickly degrades as the size of the platoon increases. In fact, performance degrades quadratically as the platoon size N increases, and would not be practical above roughly $N = 25$.

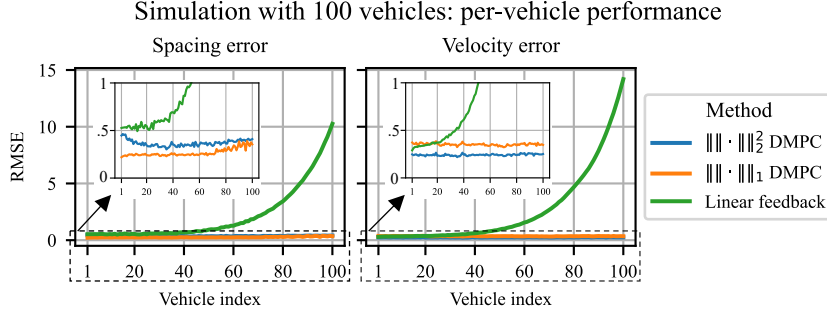


Fig. 6. Results showing the mean RMSE calculated after repeating the experiment shown in fig. 5 ten times. The zoomed in portion of each plot shows the DMPC results more clearly and the platoon size at which linear feedback begins to diverge.

Optimal control methods could be used to select better gain values, but for the sake of comparison we used the same values in simulation and on hardware.

Unlike the linear feedback controller, the DMPC controller performance does not degrade significantly as the platoon size increases up to $N = 100$ following vehicles. This is evident in the zoomed-in plots of fig. 6, where we can see the RMSE does not get worse as we move to the end of the platoon. In the spacing RMSE, however, we can see that the performance appears to be trending worse as we move further down the platoon, but this trend may not become dramatic enough until the platoon becomes much larger, potentially beyond practical platoon sizes. Finally, it is interesting to note that the better performance of the 1-norm compared to the squared 2-norm DMPC controller on hardware did not translate to simulation.

4 Conclusion

In this work, we introduced a benchmarking testbed to evaluate autonomous platooning algorithms. We introduced two DMPC algorithms and compared their performance to a baseline linear feedback algorithm. Using our benchmarking testbed, we showed that the DMPC algorithms outperform the linear feedback algorithm for a platoon of four vehicles. We performed further experiments in simulation to demonstrate that not only do the DMPC algorithms perform better on hardware, they also scale well to increases in size of the platoon.

Future work will consider how we can improve upon these methods both in theory and practice. Theoretically, there is a need to ensure the DMPC algorithms we use are string stable, and not just asymptotically stable. This may not be possible with the current formulation, but a similar formulation with a CTH policy has potential. In this work, we arbitrarily select desired distances, but future research could investigate how these distances can be rigorously determined to ensure safety (*i.e.*, no collisions) and performance (*i.e.*, small inter-vehicle spacing). Finally, future research may explore the benefits of incorporating provably safe machine learning methods and benchmarking against the control-theoretic methods used in this paper.

Acknowledgements

Research was sponsored by the DEVCOM Analysis Center and was accomplished under Cooperative Agreement Number W911NF-22-2-001. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the Army Research Office or the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein.

References

1. A. A. Alam, A. Gattami, and K. H. Johansson. Suboptimal decentralized controller design for chain structures: Applications to vehicle formations. In *IEEE Conference on Decision and Control and European Control Conference*, pages 6894–6900, Orlando, FL, USA, Dec. 2011. IEEE.
2. R. C. Coulter. Implementation of the pure pursuit path tracking algorithm. Technical report, Robotics Institute, Carnegie Mellon University, 1992.
3. S. Diamond and S. Boyd. Cvxpy: A python-embedded modeling language for convex optimization. *The Journal of Machine Learning Research*, 17(1):2909–2913, 2016.
4. A. Domahidi, E. Chu, and S. Boyd. Ecos: An socp solver for embedded systems. In *2013 European control conference (ECC)*, pages 3071–3076. IEEE, 2013.
5. W. B. Dunbar and D. S. Caveney. Distributed receding horizon control of vehicle platoons: Stability and string stability. *IEEE Transactions on Automatic Control*, 57(3):620–633, 2012.
6. M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, KDD’96, page 226–231. AAAI Press, 1996.
7. S. Feng, Y. Zhang, S. E. Li, Z. Cao, H. X. Liu, and L. Li. String stability for vehicular platoon control: Definitions and analysis methods. *Annual Reviews in Control*, 47:81–97, Mar. 2019.
8. N. Hyldmar, Y. He, and A. Prorok. A fleet of miniature cars for experiments in cooperative driving. In *2019 International Conference on Robotics and Automation (ICRA)*, pages 3238–3244, 2019.
9. S. Karaman, A. Anders, M. Boulet, J. Connor, K. Gregson, W. Guerra, O. Guldner, M. Mohamoud, B. Plancher, R. Shin, and J. Vivilecchia. Project-based, collaborative, algorithmic robotics for high school students: Programming self-driving race cars at mit. In *2017 IEEE Integrated STEM Education Conference (ISEC)*, pages 195–203, 2017.
10. M. Kloock, P. Scheffe, J. Maczjewski, A. Kampmann, A. Mokhtarian, S. Kowalewski, and B. Alrifae. Cyber-physical mobility lab: An open-source platform for networked and autonomous vehicles. In *2021 European Control Conference (ECC)*, pages 1937–1944, 2021.
11. H. Lee, J. Park, C. Koo, J.-C. Kim, and Y. Eun. Cyclops: Open platform for scale truck platooning. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 8971–8977. IEEE, 2022.

12. W. Levine and M. Athans. On the optimal error regulation of a string of moving vehicles. *IEEE Transactions on Automatic Control*, 11(3):355–361, 1966.
13. S. E. Li, Y. Zheng, K. Li, and J. Wang. An overview of vehicular platoon control under the four-component framework. In *2015 IEEE Intelligent Vehicles Symposium (IV)*, pages 286–291, 2015.
14. S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022.
15. G. J. L. Naus, R. P. A. Vugts, J. Ploeg, M. J. G. van de Molengraft, and M. Steinbuch. String-stable cacc design and experimental validation: A frequency-domain approach. *IEEE Transactions on Vehicular Technology*, 59(9):4268–4279, 2010.
16. M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam. Fltenth: An open-source evaluation environment for continuous control and reinforcement learning. In *NeurIPS 2019 Competition and Demonstration Track*, pages 77–89. PMLR, 2020.
17. T. Padir. Bright: Benchmarking research infrastructure for generalized heterogeneous teams. In *Robotics Research - The 19th International Symposium ISRR 2019, Hanoi, Vietnam, October 6-10, 2019*, volume 20 of *Springer Proceedings in Advanced Robotics*, pages 805–812. Springer, 2019.
18. J. Ploeg, S. Shladover, H. Nijmeijer, and N. van de Wouw. Introduction to the special issue on the 2011 grand cooperative driving challenge. *IEEE Transactions on Intelligent Transportation Systems*, 13(3):989–993, 2012.
19. Z. Qiang, L. Dai, B. Chen, and Y. Xia. Distributed Model Predictive Control for Heterogeneous Vehicle Platoon With Inter-Vehicular Spacing Constraints. *IEEE Transactions on Intelligent Transportation Systems*, 24(3):3339–3351, Mar. 2023.
20. T. Robinson, E. Chan, and E. Coelingh. Operating platoons on public motorways: An introduction to the sartre platooning programme. In *17th world congress on intelligent transport systems*, volume 1, page 12, 2010.
21. D. Rojas Rueda, M. J. Nieuwenhuisen, H. Khreis, and H. Frumkin. Autonomous vehicles and public health. *Annu Rev Public Health*. 2020 Apr 2; 41: 329-45, 2020.
22. P. Seiler, A. Pant, and K. Hedrick. Disturbance propagation in vehicle strings. *IEEE Transactions on Automatic Control*, 49(10):1835–1842, 2004.
23. B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd. Osqp: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.
24. D. Swaroop and J. Hedrick. String stability of interconnected systems. *IEEE Transactions on Automatic Control*, 41(3):349–357, 1996.
25. H.-S. Tan, R. Rajamani, and W.-B. Zhang. Demonstration of an automated highway platoon system. In *Proceedings of the 1998 American Control Conference. ACC (IEEE Cat. No.98CH36207)*, volume 3, pages 1823–1827 vol.3, 1998.
26. S. Tsugawa, S. Kato, and K. Aoki. An automated truck platoon for energy saving. In *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 4109–4114, 2011.
27. G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou. Aggressive driving with model predictive path integral control. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1433–1440, 2016.
28. Y. Zheng, S. Eben Li, J. Wang, D. Cao, and K. Li. Stability and scalability of homogeneous vehicular platoon: Study on the influence of information flow topologies. *IEEE Transactions on Intelligent Transportation Systems*, 17(1):14–26, 2016.
29. Y. Zheng, S. E. Li, K. Li, F. Borrelli, and J. K. Hedrick. Distributed model predictive control for heterogeneous vehicle platoons under unidirectional topologies. *IEEE Transactions on Control Systems Technology*, 25(3):899–910, 2017.